

# Replicated Counter Service: Test Report

Distributed Systems - Individual assignment (Labs 2-5) and midterm

**Author:** Murat Nazikey **Date:** 9 October 2026

**Repository:** [github.com/nazikeimurat-hash/dcc-assignment2](https://github.com/nazikeimurat-hash/dcc-assignment2)

This report documents a gRPC counter service (Part A), its Lamport-clock instrumentation (Part B) and a three-replica version with majority-acknowledged writes (Part C). Every behavioural claim below is tied to a named automated test, to the captured trace `logs/b2_trace.log`, or to the benchmark output `logs/perf_results.txt`. The full suite (11 tests) was run in one session: **11 passed in 17.99 s**.

## 1. Requirements under test

| Task  | Requirement                                                                                                                    | Verified by                                                                                                                                                                                                  |
|-------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A2    | Duplicate suppression under the lock; deadline + bounded retry with the same key                                               | <code>test_increment_applies_delta</code> , <code>test_duplicate_key_not_reapplied</code> , <code>test_retry_after_timeout_is_safe</code>                                                                    |
| A2/A1 | Get on an unknown counter returns <code>found = false</code> without raising                                                   | <code>test_get_missing_counter</code>                                                                                                                                                                        |
| A3    | Two concurrent clients x 1,000 increments give exactly 2,000                                                                   | <code>test_concurrent_increments_exact</code>                                                                                                                                                                |
| B1    | Lamport clock: tick before local events, $\max(\text{own}, \text{received}) + 1$ on receive, counter attached to every message | <b>No automated test.</b> Checked by reading the trace produced by <code>scenario_b2.py</code> (Appendix A): every <code>RECV</code> line satisfies $L = \max(\text{previous } L, \text{received } L) + 1$ . |
| C1    | Write commits only with at least 2 of 3 acknowledgements; replicas converge                                                    | <code>test_majority_commit_two_acks</code> , <code>test_no_commit_below_majority</code> , <code>test_replicas_converge</code> , <code>test_crash_mid_request_still_commits</code>                            |
| C3    | Crash, duplication and timeout-with-retry scenarios assert invariants                                                          | <code>test_crash_mid_request_still_commits</code> , <code>test_request_duplication_moves_value_once</code> , <code>test_induced_timeout_retry_moves_once</code>                                              |
| C4    | Median and p95 latency for four configurations                                                                                 | <code>perf_benchmark.py</code> (Section 5)                                                                                                                                                                   |

## 2. Test environment

| Item             | Value                                                                                                                                                                               |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Operating system | Windows 11 Home, version 24H2 (OS build 26100.6584), 64-bit (AMD64)                                                                                                                 |
| Python           | 3.14.8, in a virtual environment ( <code>.venv</code> )                                                                                                                             |
| Key packages     | <code>grpcio</code> 1.84.0, <code>grpcio-tools</code> 1.84.0, <code>protobuf</code> 7.36.2, <code>pytest</code> 9.1.1 (full list: Appendix B and <code>logs/pip_freeze.txt</code> ) |
| Topology         | All replicas, clients and the benchmark ran on one machine over localhost. Tests use ephemeral ports (port 0) or free ports found at run time.                                      |
| Commands         | <code>python -m pytest tests -v</code> ; <code>python perf_benchmark.py</code> ; <code>python scenario_b2.py</code>                                                                 |

## 3. Test-case table

| ID | Test (purpose)                                                                            | Expected                                                                                             | Actual                    | Result |
|----|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------|--------|
| T1 | <code>test_increment_applies_delta</code> : one Increment applies exactly its delta       | <code>Increment(x, 7)</code> returns 7, <code>was_duplicate = false</code> , <code>Get(x) = 7</code> | As expected               | PASS   |
| T2 | <code>test_duplicate_key_not_reapplied</code> : same key re-sent                          | Second reply 5, <code>was_duplicate = true</code> , <code>Get(x) = 5</code> (not 10)                 | As expected               | PASS   |
| T3 | <code>test_concurrent_increments_exact</code> : 2 threads x 1,000 increments              | Final value exactly 2000                                                                             | 2000                      | PASS   |
| T4 | <code>test_get_missing_counter</code> : unknown counter id                                | <code>found = false</code> , no exception                                                            | As expected               | PASS   |
| T5 | <code>test_retry_after_timeout_is_safe</code> : server delayed 1 s, client deadline 0.3 s | First call raises <code>DEADLINE_EXCEEDED</code> ; retry with same key succeeds; counter = 1         | As expected (counter = 1) | PASS   |
| T6 | <code>test_majority_commit_two_acks</code> : one replica stopped                          | <code>committed = true</code> , <code>acks = 2</code>                                                | As expected               | PASS   |

| ID  | Test (purpose)                                                | Expected                                                                 | Actual                                                                | Result |
|-----|---------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------|--------|
| T7  | test_no_commit_below_majority: two replicas stopped           | committed = false, acks = 1; surviving replica not reported as committed | committed = false, acks = 1 (live replica holds x = 1, see Section 6) | PASS   |
| T8  | test_replicas_converge: 20 quorum writes                      | All three replicas hold {x: 20}                                          | As expected                                                           | PASS   |
| T9  | test_crash_mid_request_still_commits (failure injection)      | No exception; committed = true; acks = 2                                 | As expected                                                           | PASS   |
| T10 | test_request_duplication_moves_value_once (failure injection) | Second write was _duplicate = true; every replica value = 1              | As expected                                                           | PASS   |
| T11 | test_induced_timeout_retry_moves_once (failure injection)     | committed = true, acks = 3, every replica {x: 1}                         | As expected                                                           | PASS   |

Summary: 11 executed, 11 passed, 0 failed (pytest output: 11 passed in 17.99 s).

## 4. Failure-injection results

| Scenario                  | Injected fault                                                                                                                                                                        | Observed invariant                                                           | Evidence                                                                 |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Replica crash mid-request | Three replicas started as separate processes (subprocess.Popen); replica-1 started with --delay-ms 800 so the write is still in flight; proc.kill() sent 0.2 s after the write begins | No exception reaches the caller; committed = true with exactly 2 acks        | tests/test_failures.py::test_crash_mid_request_still_commits PASSED      |
| Request duplication       | Same idempotency key sent twice through the quorum client to three replica processes                                                                                                  | Second result has was_duplicate = true; each replica reports value 1         | tests/test_failures.py::test_request_duplication_moves_value_once PASSED |
| Induced timeout + retry   | Replica-2 delay set to 1.0 s while the client deadline is 0.4 s; delay removed after 0.5 s so the retry is fast                                                                       | Write commits with 3 acks and every replica holds x = 1 (moved exactly once) | tests/test_failures.py::test_induced_timeout_retry_moves_once PASSED     |

Note on evidence: the test fixture sends replica stdout to DEVNULL, so the evidence is the pytest result line for each scenario rather than a replica log excerpt. The timeout scenario injects its fault in-process (setting the servicer delay) instead of through a --fault flag.

## 5. Performance (Task C4)

**Method.** perf\_benchmark.py starts three server processes, performs 50 warm-up increments, then runs 2,000 Increment calls per configuration (split evenly across 1 or 16 client threads). Latency is measured with time.perf\_counter() around each client call (for the quorum path: until the client has collected all three outcomes). The median is the statistical median; p95 is the value at 1-based rank ceil(0.95 n) of the sorted latencies. One run per configuration.

| Configuration                   | Median latency (ms) | p95 latency (ms) | Requests | Throughput (req/s) |
|---------------------------------|---------------------|------------------|----------|--------------------|
| Single replica, 1 client        | 0.78                | 1.18             | 2000     | 1242               |
| Single replica, 16 clients      | 7.21                | 10.08            | 2000     | 2146               |
| Quorum (3 replicas), 1 client   | 1.20                | 2.21             | 2000     | 698                |
| Quorum (3 replicas), 16 clients | 16.92               | 22.66            | 2000     | 907                |

**Interpretation.** The quorum path sends every write to three replicas and, in this implementation, waits for all three outcomes, so its latency is set by the slowest replica plus the extra client-side work of three RPCs. All three replicas also share one machine and one CPU, so each write costs about three times the server work; with one client the median rises from 0.78 to 1.20 ms, and with 16 clients from 7.21 to 16.92 ms while throughput falls from 2146 to 907 req/s. In absolute terms the overhead is a few to about ten milliseconds (p95 22.66 ms at 16 clients), which is acceptable for a like counter, where losing acknowledged writes on a replica crash is the larger risk. The numbers come from one run on localhost and do not include real network delay.

## 6. Known limitations

- **Partial writes (C1 anomaly).** With only one acknowledgement the client reports failure, but that replica has already applied the write (T7 asserts this). Nothing rolls it back, so replicas can diverge and a read from that replica shows a value that was never committed.

- **Single-replica reads.** Get is served by the first replica that answers, so it can return a stale value (a replica that missed a write) or an uncommitted one. Reads are not linearizable.
- **No durability.** Counter values and the deduplication store are in memory. A restarted replica comes back empty, so committed writes can be lost if enough replicas restart.
- **B1 is not covered by an automated test.** Lamport rules were verified by inspecting the trace, not by an assertion. Also, ReplicatedClient creates one CounterClient (and so one Lamport clock) per replica, so a quorum write is not a single logical process in the trace.
- **Quorum latency.** The client waits for all replies rather than returning after the second acknowledgement, and a dead replica costs the full retry back-off (0.2 + 0.4 + 0.8 s) per write.
- **Benchmark validity.** Single run, one machine, loopback only, no variance reported; servers print one log line per request, which inflates absolute latencies. Only a --delay-ms fault flag exists (no --fault / drop-after-recv).

## 7. Design notes required by the assignment

### A2: RPC semantics

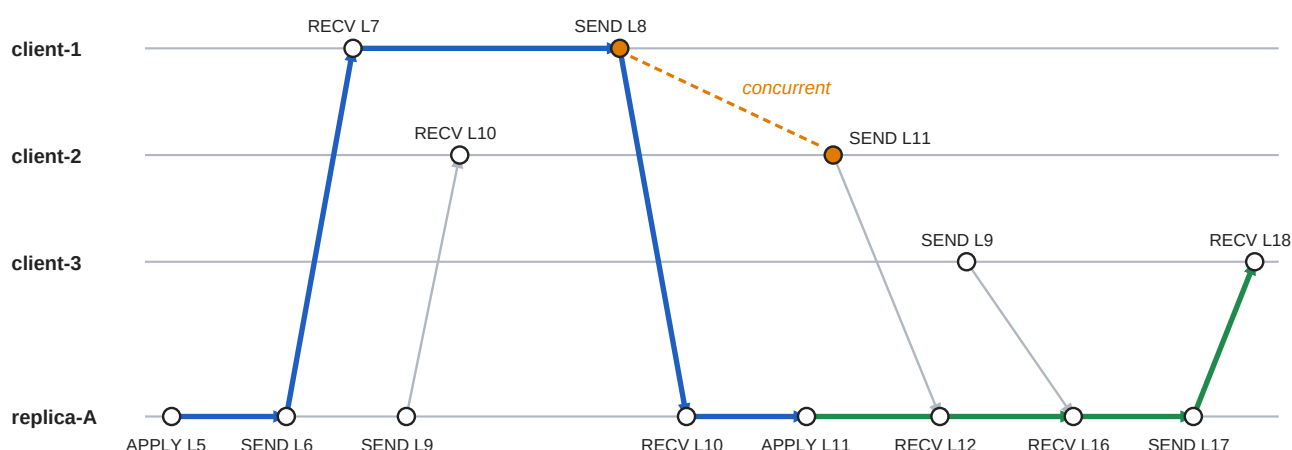
The server gives **at-most-once execution of the mutation per idempotency key**. Increment checks the deduplication store (`_seen`) and, if the key is new, updates `_values` and records the result, all inside one `threading.Lock`, so two concurrent retries cannot both pass the check. Because the client generates the key once per logical operation and reuses it on every retry (deadline 2 s, at most 3 retries, back-off 0.2/0.4/0.8 s), a retry after a lost reply returns the stored result with `was_duplicate = true` instead of adding the delta again. (T2, T5, T10, T11.) The same lock is the synchronization mechanism for A3 (T3).

### C1: partial-write anomaly and Get consistency

If one replica acknowledges, the client reports failure although that replica applied the write, so a failed write is partly visible. A consensus protocol such as Raft avoids this: a leader appends the entry to its log, replicates it, and only after a majority has stored it marks it committed and applies it to the state machine; followers do not apply uncommitted entries, and a new leader can discard them. Reading Get from any single replica risks a stale read (the replica missed a write) or an uncommitted read (the partial write above).

## 8. Task B2: ordering analysis

Scenario (scenario\_b2.py): client-1 increments x twice, client-2 increments y twice, client-3 issues three Gets on x, against replica-A. The trace has 32 lines (Appendix A). Lines in the file are in the order they were written, not in causal order, so causality is argued only from local order and message edges. The diagram shows the events used below.



**Figure 1.** Happened-before diagram (subset). Blue: chain proving  $\text{APPLY}(L=5) \rightarrow \text{APPLY}(L=11)$ . Green: chain proving  $\text{APPLY}(L=11) \rightarrow \text{client-3 RECV}(L=18)$ . Orange: concurrent pair. Gray arrows: other messages.

### Causally ordered pair 1: $\text{APPLY } x=1 (L=5)$ happened-before $\text{APPLY } x=2 (L=11)$

replica  $\text{APPLY } (L=5)$  is local-before replica  $\text{SEND IncrementReply } (L=6)$ ; that  $\text{SEND}$  is message-before client-1  $\text{RECV } (L=7)$ ;  $\text{RECV } (L=7)$  is local-before client-1  $\text{SEND } (L=8)$ ; that  $\text{SEND}$  is message-before replica  $\text{RECV } (L=10)$ , which is local-before  $\text{APPLY } x=2 (L=11)$ .

### **Causally ordered pair 2: APPLY x=2 (L=11) happened-before client-3 RECV GetReply value=2 (L=18)**

replica APPLY (L=11) is local-before replica RECV Get (L=16, received L=9 from client-3), which is local-before replica SEND GetReply (L=17), which is message-before client-3 RECV (L=18). So the value 2 that client-3 saw was necessarily produced by the earlier write.

### **Concurrent pair: client-1 SEND (L=8) and client-2 SEND (L=11)**

No chain of local-order and message edges connects them in either direction. client-2's SEND (L=11) follows client-2 RECV (L=10), which received replica SEND (L=9); the replica only received client-1's message (L=8) at its own RECV (L=10), after that SEND. Their values differ ( $8 < 11$ ), but Lamport clocks only guarantee that  $a \rightarrow b$  implies  $L(a) < L(b)$ ; the converse does not hold. The smaller value does not mean client-1 acted first: L counts events per process and is merged with  $\max()$  only through messages, so it says nothing about wall-clock or causal order between independent processes. In the same trace, three events (the first SENDs of client-1, client-2 and client-3) all have  $L=1$ .

### **Limitation removed by vector clocks**

With Lamport clocks,  $L(a) < L(b)$  cannot distinguish 'a happened-before b' from 'a and b are concurrent'. Vector clocks keep one counter per process, so  $V(a) < V(b)$  holds exactly when a happened-before b, and incomparable vectors identify concurrent events.

## **9. Reflection questions**

**1. Request lost vs. reply lost.** After a timeout the client only learns that no reply arrived within the deadline: the request may never have reached the server, may have been executed with the reply lost, or may be slow. Nothing in the failure distinguishes these. The idempotency key makes the ambiguity harmless: the server stores the result under the key, so a retry either executes the operation for the first time or returns the stored result, and the counter moves exactly once (T2, T5, T11).

**2. Concurrent events with different Lamport values.** client-1 SEND (L=8) and client-2 SEND (L=11) are concurrent (Section 8). The values depend on how many events each process had and on messages merged via  $\max()$ ; with no message path between them,  $8 < 11$  carries no information about which happened first.

**3. Tolerated failures.** One replica can crash (or be slow or unreachable): the remaining two still acknowledge, so writes keep committing (T6, T9) and every committed write was acknowledged by two replicas, so at least one live replica has it. Two failed replicas break liveness: no write can commit (T7). Safety can also break if the two crashed replicas are exactly the two that acknowledged an earlier committed write and state is not persisted, or if Get reads from a replica that missed the write. In safety/liveness terms: one failure preserves both (apart from the stale-read caveat); two failures lose liveness and may lose safety.

**4. Surviving restarts.** The smallest change is to make each replica durable: append every applied operation (counter id, delta, idempotency key) to a write-ahead log on disk and flush it before acknowledging, then replay the log at start-up to rebuild the counters and the deduplication store. This anticipates the persistent replicated log of Raft (Week 6).

## Appendix A: event-log excerpt (logs/b2\_trace.log, 32 lines)

```
[client-1] SEND Increment(counter=x, delta=1) L=1
[client-2] SEND Increment(counter=y, delta=1) L=1
[client-3] SEND Get(counter=x) L=1
[replica-A] RECV Increment(counter=x, delta=1) L=2 (received L=1)
[replica-A] RECV Get(counter=x) L=3 (received L=1)
[replica-A] RECV Increment(counter=y, delta=1) L=4 (received L=1)
[replica-A] APPLY counter=x -> 1 L=5
[replica-A] SEND IncrementReply(new_value=1) L=6
[replica-A] SEND GetReply(value=1, found=True) L=7
[replica-A] APPLY counter=y -> 1 L=8
[client-1] RECV IncrementReply(new_value=1) L=7 (received L=6)
[client-3] RECV GetReply(value=1) L=8 (received L=7)
[replica-A] SEND IncrementReply(new_value=1) L=9
[client-1] SEND Increment(counter=x, delta=1) L=8
[client-2] RECV IncrementReply(new_value=1) L=10 (received L=9)
[client-2] SEND Increment(counter=y, delta=1) L=11
[replica-A] RECV Increment(counter=x, delta=1) L=10 (received L=8)
[replica-A] APPLY counter=x -> 2 L=11
[replica-A] RECV Increment(counter=y, delta=1) L=12 (received L=11)
[replica-A] SEND IncrementReply(new_value=2) L=13
[replica-A] APPLY counter=y -> 2 L=14
[client-1] RECV IncrementReply(new_value=2) L=14 (received L=13)
[replica-A] SEND IncrementReply(new_value=2) L=15
[client-2] RECV IncrementReply(new_value=2) L=16 (received L=15)
[client-3] SEND Get(counter=x) L=9
[replica-A] RECV Get(counter=x) L=16 (received L=9)
[replica-A] SEND GetReply(value=2, found=True) L=17
[client-3] RECV GetReply(value=2) L=18 (received L=17)
[client-3] SEND Get(counter=x) L=19
[replica-A] RECV Get(counter=x) L=20 (received L=19)
[replica-A] SEND GetReply(value=2, found=True) L=21
[client-3] RECV GetReply(value=2) L=22 (received L=21)
```

## Appendix B: installed packages

Packages installed into the virtual environment by `pip install grpcio grpcio-tools pytest` (from the pip output; the complete pip freeze file is `logs/pip_freeze.txt` in the repository):

```
colorama==0.4.6
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.1
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0
```